

Data Structures and Algorithms: Quick Reference Notes

Arrays

An array is a linear data structure that stores elements of the same type in contiguous memory locations. Each element can be accessed directly using its index, which makes read operations run in constant time, $O(1)$. Inserting or deleting an element in the middle of an array requires shifting subsequent elements, giving those operations a worst case time complexity of $O(n)$. Arrays are the foundation for many other data structures, including stacks, queues, and hash tables built with open addressing.

Linked Lists

A linked list is a linear data structure where each element, called a node, stores a value along with a reference to the next node in the sequence. Unlike arrays, linked lists do not require contiguous memory, so insertion and deletion at a known position take $O(1)$ time once the position is found. However, accessing an arbitrary element requires traversing the list from the head, which takes $O(n)$ time. Singly linked lists store one forward pointer per node, while doubly linked lists store both forward and backward pointers, allowing traversal in either direction.

Stacks

A stack is a linear data structure that follows the Last In First Out, or LIFO, principle. Elements are added and removed from the same end, called the top of the stack. The two primary operations are push, which adds an element, and pop, which removes the most recently added element. Both operations run in $O(1)$ time when implemented with an array or a linked list. Stacks are commonly used to implement function call management, undo features in editors, and to evaluate expressions in postfix or prefix notation.

Queues

A queue is a linear data structure that follows the First In First Out, or FIFO, principle. Elements are added at the rear and removed from the front. The two primary operations are enqueue, which adds an element to the rear, and dequeue, which removes an element from the front. A circular queue reuses empty space at the beginning of an array once elements are dequeued, avoiding the need to shift elements. Queues are widely used in task scheduling, breadth first search traversal, and handling requests in the order they arrive.

Binary Search

Binary search is an efficient algorithm for finding an element in a sorted array. It repeatedly divides the search range in half by comparing the target value with the middle element. If the target is smaller, the search continues in the left half; if larger, it continues in the right half. This halving approach gives binary search a time complexity of $O(\log n)$, which is significantly faster than a linear scan for large datasets. Binary search requires the input array to be sorted beforehand.

Sorting Algorithms

Sorting algorithms arrange elements of a collection in a particular order, typically ascending or descending. Bubble sort and insertion sort are simple approaches with $O(n^2)$ time complexity, suitable mainly for small or nearly sorted datasets. Merge sort and quicksort use a divide and conquer strategy to achieve an average time complexity of $O(n \log n)$. Merge sort guarantees $O(n \log n)$ performance in all cases but requires additional memory, while quicksort is typically faster in practice but can degrade to $O(n^2)$ in the worst case.

Time and Space Complexity

Time complexity describes how the running time of an algorithm grows as the size of its input increases, usually expressed using Big O notation. Space complexity describes how much additional memory an algorithm requires relative to the input size. Common time complexities, from fastest to slowest growth, include $O(1)$ constant time, $O(\log n)$ logarithmic time, $O(n)$ linear time, $O(n \log n)$ linearithmic time, and $O(n^2)$ quadratic time. Understanding these complexities helps in choosing the right algorithm for a given problem size and performance requirement.

Binary Trees

A binary tree is a hierarchical data structure in which each node has at most two children, commonly referred to as the left child and the right child. A binary search tree, or BST, additionally maintains the property that all values in a node's left subtree are smaller than the node's value, and all values in the right subtree are larger. This property allows search, insertion, and deletion operations to run in $O(\log n)$ time on a balanced tree, though these operations degrade to $O(n)$ on an unbalanced tree that resembles a linked list.